Mikrocomputertechnik 1

Einheit 7: Performance & Pipelining

Prof. Dr.-Ing. Daniel Klünder

Folie 1 / 85

Performance & Pipelining

Notizen

Einheit 5 und 6 haben Datenpfad und Steuerwerk erklärt. Einheit 7 fragt nach der Laufzeit. Performance ist hier die überprüfbare Programmlaufzeit beziehungsweise der Durchsatz. Pipelining überlappt mehrere Befehle zeitlich. Die Lernziele sind: die CPU-Zeit rechnen, ein Zeitdiagramm lesen und einen Hazard nachvollziehen. CPU bedeutet Central Processing Unit.
Kontrollfrage: Sagt ein korrektes Ergebnis allein die Laufzeit? Nein. Die nächste Folie motiviert das Fließband.

Folie 2 / 85

Fließbandarbeit im Prozessor

- Herzlich willkommen zur siebten Vorlesung in Mikrocomputertechnik 1
- Bisher: funktionierender, aber architektonisch langsamer Single-Cycle-Prozessor
- Heute: Fließbandarbeit (Pipelining) als Architektur-Revolution
- Mehrere Befehle gleichzeitig im Datenpfad
- Stolpersteine: Data Hazards und Control Hazards



Notizen

Ein Single-Cycle-Datenpfad führt einen unterstützten Befehl in einem langen Takt aus. Der Takt wird vom längsten Pfad begrenzt. Das ist verständlich, aber nicht automatisch langsam für jede Aufgabe. Mehrere gleichzeitig bearbeitete Instruktionen sind hier nicht mehrere gleichzeitig bestätigte Ergebnisse pro Takt. Ein Data Hazard betrifft eine benötigte Datenabhängigkeit, ein Control Hazard den noch unbekannten nächsten Befehlsfluss.

Kontrollfrage: Ist Single Cycle eine Sackgasse? Nein, ein einfaches Modell. Die Agenda verbindet Rechnung und Korrektheit.

Folie 3 / 85

Agenda für den heutigen Tag

1. Performance-Grenzen — warum Single-Cycle eine Sackgasse ist
2. Die 5-stufige RISC-V-Pipeline — Datenpfad zersägen, Pipeline-Register
3. Data Hazards & Forwarding — Variablen-Abhängigkeiten (RAW)
4. Load-Use Hazards & Stalls — wenn Forwarding nicht reicht
5. Control Hazards — Sprünge, Flushes, Branch Prediction
6. Praxis (Ripes) — Hazards und Pipeline-Bubbles live



Notizen

Ziele: CPU-Zeit berechnen, IF, ID, EX, MEM und WB unterscheiden, RAW erkennen, Forwarding begründen, einen Load-Use-Stall erklären und falsche Branch-Folgebefehle erkennen. IF ist Instruction Fetch, ID Instruction Decode, EX Execute, MEM Memory Access, WB Write Back. RAW bedeutet Read After Write. RISC-V ist die Befehlssatzarchitektur. Fünf Stufen sind unser Modell, keine ISA-Vorschrift. Für einfache Systeme können andere Entwurfsziele gelten.
Kontrollfrage: Schreibt die ISA fünf Stufen vor? Nein. Zuerst reaktivieren wir den Single-Cycle-Ausgangspunkt.

Folie 4 / 85

Rückblick: Der Single-Cycle-Datenpfad

- Bisher: jeder Befehl braucht exakt einen Taktzyklus
- Phasen Fetch, Decode, Execute, Memory, Write-Back aneinandergereiht
- Steuerwerk dirigiert fehlerfrei
- Logik korrekt — Silizium-Nutzung miserabel
- Warum verwerfen Apple, Intel und AMD dieses Design?

```
Ein Design ist nur dann gut, wenn es nicht nur logisch korrekt,
sondern auch effizient ist.
```

Notizen

Die fünf Bereiche Fetch, Decode, Execute, Memory und Write Back arbeiten hier noch innerhalb eines langen Zyklus für dieselbe Instruktion. Sie sind noch keine fünf getakteten Stufen. Fetch holt den Befehl, Decode liest Register und steuert, Execute rechnet, Memory greift auf Daten zu, Write Back schreibt ein Register. Einfachheit, Fläche und Energie können ebenfalls Entwurfsziele sein. Konkrete Werte behaupten

wir dafür nicht.
Kontrollfrage: Sind die fünf Kästen schon Pipeline-Takte? Nein. Der kritische Pfad setzt die Taktperiode.

Folie 5 / 85

Die Timing-Falle (Refresher)

- Taktfrequenz diktiert vom kritischen Pfad
- Ein Takt muss den längsten relevanten Signalpfad abdecken, nicht nur ein einzelnes Gatter
- Langsamster Befehl: `lw` — nutzt alle Komponenten und setzt die Uhr
- Fazit: extrem langer Takt drosselt die gesamte CPU

```
Single-Cycle-Takt (kritischer Pfad):
Load Word: |====================|  (gibt den Takt vor)
Add:       |========|............  (wartet am Ende)
           ^ fertig   ^ verschwendete Zeit
```

Notizen

Der kritische Pfad ist der längste relevante Signalweg einschließlich der Zustandsübergangsbedingungen, nicht das langsamste einzelne Gatter. lw, Load Word, durchläuft in diesem Datenpfad Befehlsspeicher, Registerlesen, ALU, Datenspeicher und Write-Back-Auswahl. ALU bedeutet Arithmetic Logic Unit. Das ist ein langer Pfad dieses Modells, kein Nachweis für jede CPU. Die Taktperiode muss diesen Weg plus den effektiven Registeroverhead aufnehmen.
Kontrollfrage: Reicht das langsamste Gatter als Taktgrenze? Nein. Schnellere Befehle nutzen denselben langen Takt.

Folie 6 / 85

Ressourcen-Verschwendung

- Was passiert bei einem schnellen Befehl wie `add`?
- ALU und Write-Back sind nach einem Bruchteil des Takts fertig
- Fetch/Decode/ALU liegen den Rest des Zyklus still
- Massive Rechenleistung geht durch Warten auf die nächste Flanke verloren

Notizen

Eine kombinatorische Stufe kann nach dem Einschwingen stabil sein. Das ist nicht automatisch abgeschalteter Strom. Entscheidend ist die ungenutzte Zeit, bevor die nächste Instruktion aufgenommen wird. Die ALU rechnet in diesem Bild nur einen Teil der Periode. Dieselben qualitativen Wege wie zuvor gelten, ohne erfundene Nanosekunden.
Kontrollfrage: Liegen Fetch und Decode elektrisch still, sobald ihr Teil fertig ist? Nicht zwingend. Die Formel trennt die Stellgrößen.

Folie 7 / 85

Wie steigert man Performance?

- Unbestechliche Formel der Rechnerarchitektur:
- CPU-Zeit = Befehlsanzahl · CPI · Taktzeit
- Befehlsanzahl: dynamisch ausgeführte echte Maschineninstruktionen, nicht Dateigröße oder Assemblerzeilen
- CPI (Cycles per Instruction): mittlere Takte pro Befehl
- Taktzeit: Dauer eines Clock-Ticks (Nanosekunden)

Notizen

CPU-Zeit gleich N mal CPI mal Taktperiode. N ist die Anzahl dynamisch abgeschlossener Maschineninstruktionen, nicht die Quellzeilenzahl. CPI bedeutet Cycles per Instruction, mittlere Takte pro Instruktion. Die Einheiten ergeben Sekunden. CPI gleich 1 beweist allein keine hohe Leistung, weil der Takt lang sein kann. Compiler können N ändern. Zwei Systeme werden auf dieselbe Aufgabe bezogen. Schleifen erhöhen N durch Wiederholung.
Kontrollfrage: Zählt eine Quellzeile als eine dynamische Instruktion? Nicht allgemein. Eine Stationsfolge führt zum Pipelineprinzip.

Folie 8 / 85

Die Waschsalon-Metapher

- Pipelining intuitiv: kurz weg von Hardware, rein in den Waschsalon
- Vier Körbe Schmutzwäsche, vier Schritte je 30 Minuten:
- Waschen → Trocknen → Falten → Schrank
- Eine komplette Ladung dauert immer exakt 2 Stunden



Notizen

Vier Stationen brauchen im Modell je 30 Minuten. Ein Korb braucht 120 Minuten, wenn keine Zusatzzeit zwischen den Stationen anfällt. Das ist ein Lehrbild, kein gemessenes Waschsalon-Timing. Die Stationen brauchen unabhängige Kapazität, auch beim Falten. Die Latenz eines Korbs ist zwei Stunden. Die Gesamtzeit mehrerer Körbe ist noch offen.
Kontrollfrage: Wie lange braucht ein Korb? Zwei Stunden. Zuerst ohne Überlappung.

Folie 9 / 85

Sequenzielle Arbeit (Single-Cycle)

- Wie ein Single-Cycle-Prozessor wäscht:
- Korb 1 komplett durch alle Stationen
- Erst wenn Korb 1 im Schrank liegt, startet Korb 2
- Vier Körbe: $4 \times 2\,\text{h} = 8\,\text{h}$
- Die Waschmaschine steht fast die ganze Zeit still

```
Timeline sequenziell:
[ Korb 1: Waschen -> Trocknen -> Falten -> Schrank ]
|<-------------- 2 Stunden --------------->|
  ^ erst hier startet Korb 2
```

Notizen

Vier Körbe mal vier Schritte mal 30 Minuten sind 480 Minuten, also acht Stunden. Jeder Korb belegt eine Station nur in seinem Abschnitt. Die übrigen Stationen warten. Das Bild ist keine Energie- oder Flächenaussage des Prozessors.
Kontrollfrage: Wie viele Minuten sind vier Körbe sequenziell? 480. Überlappung startet den nächsten Korb an der freien ersten Station.

Folie 10 / 85

Pipelined Waschen (Fließband)

- Intelligente Variante: Fließband (Pipeline)
- Sobald Korb 1 in den Trockner wandert, geht Korb 2 in die Waschmaschine
- Alle vier Stationen arbeiten gleichzeitig an vier Körben
- Vier Körbe: nur noch 3,5 Stunden (statt 8)

```
Pipelined (vereinfacht):
Zeit:     0.5h   1.0h   1.5h   2.0h   2.5h   3.0h   3.5h
Korb 1:   Wash   Dry    Fold   Shelf
Korb 2:          Wash   Dry    Fold   Shelf
Korb 3:                 Wash   Dry    Fold   Shelf
Korb 4:                        Wash   Dry    Fold   Shelf
```

Notizen

Sieben Halbstunden-Abschnitte: der erste Korb endet nach vier, der vierte nach sieben. Sieben mal 30 Minuten sind 3,5 Stunden. Der Gesamt-Speedup ist 8 geteilt durch 3,5, etwa 2,29. Der stationäre Durchsatz, ein Korb je 30 Minuten, ist eine andere Größe als dieser endliche Gewinn. Füllung und Leerung erklären, warum es nicht vier mal so schnell für genau vier Körbe ist.
Kontrollfrage: Ist der Gesamtgewinn Faktor 4? Nein, etwa 2,29. Die nächste Folie trennt Latenz und

Durchsatz.

Folie 11 / 85

Latenz vs. Durchsatz

- Im Waschsalonmodell bleibt die Latenz eines Korbs zwei Stunden
- Pipeline-Register und ungleiche Stufen können die Einzellatenz eines Befehls erhöhen
- Das Lehrbeispiel später: 820 ps Single Cycle, 1350 ps Pipeline-Latenz
- Der Durchsatz kann trotzdem steigen

```
Latenz    = Wie lange dauert EIN Befehl von Anfang bis Ende?
Durchsatz = Wie viele Befehle werden pro Sekunde fertiggestellt?
```

Notizen

Im Waschsalon bleibt die Latenz eines Korbs zwei Stunden. Das gilt für dieses Modell ohne Zusatzaufwand. Pipeline-Register, ungleiche Stufen und Wartezyklen können die Einzellatenz eines Befehls erhöhen. Latenz ist die Dauer eines Auftrags, Durchsatz die Fertigstellungen pro Zeit. Das CPU-Lehrbeispiel zeigt später 820 gegen 1350 Pikosekunden.
Kontrollfrage: Verkürzt Pipelining immer die Einzellatenz? Nein. Der Durchsatz wird als Nächstes quantifiziert.

Folie 12 / 85

Der Durchsatz (Throughput)

- Magie des Pipelining: Taktrate des Outputs
- Sequenziell: alle 2 Stunden ein fertiger Korb
- Pipeline (voll beladen): alle 30 Minuten ein fertiger Korb
- Durchsatz vervierfacht
- Genau so werden Computer und Grafik flüssig

Notizen

Throughput ist der Durchsatz. Stationär kommt ein Korb je 30 Minuten statt je 120 Minuten, also die vierfache Rate. Für vier Körbe ist der Gesamt-Speedup trotzdem etwa 2,29. Computer und Grafik sind nur eine Motivation, keine vereinfachte GPU-Messung. GPU würde Graphics Processing Unit bedeuten, falls der Begriff fällt.
Kontrollfrage: Ist die vierfache Rate der Gesamt-Speedup? Nein. Die Stationen werden auf den Prozessor übertragen.

Folie 13 / 85

Transfer auf den Prozessor

- Wäschestationen → fünf RISC-V-Ausführungsphasen
- IF: Instruction Fetch (Befehl aus dem IM)
- ID: Instruction Decode (Decoder & Register File)
- EX: Execute (ALU)
- MEM: Memory (Daten-RAM)
- WB: Write-Back (Ergebnis ins Register)



Notizen

IF holt aus dem Instruction Memory, kurz IM. ID liest die Registerbank, das Register File. EX nutzt die ALU. MEM kann den Datenspeicher, RAM, nutzen. WB schreibt zurück. add durchläuft MEM, ohne zwingend Daten zu lesen. Ein Store hat in WB kein Register-Write-Back. Nicht jeder Befehl nutzt in jeder Stufe jede Komponente. Fünf Stufen sind trotzdem der gemeinsame Taktplan.
Kontrollfrage: Schreibt jeder WB-Eintrag ein Register? Nein. Der nächste Abschnitt setzt die Stufengrenzen um.

Folie 14 / 85

Die 5-stufige RISC-V-Pipeline

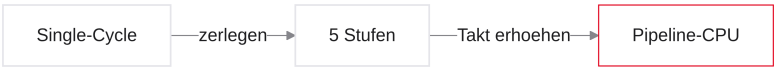
Notizen

Die fünfstufige Pipeline ist eine mögliche Mikroarchitektur für RISC-V, keine Vorschrift der ISA, der Instruction Set Architecture. Bisher war die Stationsmetapher. Jetzt werden Zeitgrenzen echte Speicherzustände. Die Frage lautet, wie Befehl, Daten und Steuerung nach jedem Takt zusammenbleiben.
Kontrollfrage: Gehört IF/ID zur ISA? Nein, zum Modell. Die nächste Folie skizziert den Umbau.

Folie 15 / 85

Den Prozessor physisch umbauen

- Säge ansetzen: Datenpfad in Stufen zerlegen
- Schnellerer Takt rettet die Performance
- Isolations-Problem: Signal-Chaos ohne Barrieren
- Pipeline-Register als Schotten
- Idealer Speedup: Faktor 5



Notizen

Ein Ideal-Speedup von 5 braucht ausgeglichene Stufen, vernachlässigbaren Overhead, lange Folgen und keine Hazards. Das ist kein garantierter Gewinn. Die Säge ist eine Metapher für eingefügte Register. Danach kommen Forwarding, Stall und Flush als neue Kontrollfunktionen. Signal-Chaos meint eine falsche zeitliche Zuordnung, nicht einen elektrischen Zusammenstoß.
Kontrollfrage: Folgt Faktor 5 aus fünf Schnitten? Nein. Die fünf Bereiche werden benannt.

Folie 16 / 85

Den Datenpfad zersägen

- Vertikale Schnitte durch den Patterson/Hennessy-Schaltplan
- Fünf Hardware-Inseln:
- IF: PC und Befehlsspeicher
- ID: Register File und Control Unit
- EX: Haupt-ALU
- MEM: Daten-RAM
- WB: Write-Back-MUX vor dem Register File
- Registergrenzen und später Forwarding, Stall und Flush verändern die Mikroarchitektur
- Fünf Stufen sind eine Modellentscheidung, keine RISC-V-Vorschrift

Notizen

Die Schnitte ändern die Mikroarchitektur. Pipeline-Register und später Forwarding, Stall und Flush sind zusätzliche Logik. IF enthält PC und Befehlsspeicher. Der PC, Program Counter, ist die Befehlsadresse. ID enthält Registerlesen und Steuerung. Der Schreibport derselben Registerbank liegt bei WB. EX enthält die ALU, MEM den Datenspeicher, WB die Schreibauswahl. Information ist ein Signalzustand, kein Strom, der von links nach rechts fließt.
Kontrollfrage: Bleibt die Gatterlogik unverändert? Nein. Die Grenzen verändern den kritischen Pfad.

Folie 17 / 85

Ein viel schnellerer Takt

- Der Takt folgt der längsten Stufe plus effektivem Registeroverhead
- Lehrwerte: Single Cycle 820 ps, Pipeline-Takt 270 ps
- Das sind Modellrechnungen, keine gemessenen 800 MHz oder 4 GHz



Notizen

Die Pipeline-Taktperiode ist die längste Stufenlogik plus ein effektiver Registeroverhead von 20 Pikosekunden. Die Stufen sind 200, 100, 150, 250 und 100 Pikosekunden. Das Maximum ist 250, plus 20 ergibt 270. Der Single-Cycle-Pfad ist 800 plus 20 gleich 820. Weniger Gatter je Stufe bedeuten nicht gleiche Laufzeiten. Ein Gigahertz ist eins geteilt durch die Periode. 270 Pikosekunden sind ein Modellwert, keine gemessenen 4 Gigahertz.
Kontrollfrage: Ist 270 ps gleich 4 GHz? Nein. Reine Schnittlinien speichern noch nichts.

Folie 18 / 85

Warum Zustandsgrenzen nötig sind

- Ohne Schutz: Signal-Chaos nach dem Zersägen
- Kombinatorik fließt ungebremst weiter
- ALU-Ergebnis rutscht in MEM, bevor der nächste Tick kommt
- Befehle krachen ineinander (Auffahrunfall)
- Dicke Wände zwischen den Inseln nötig

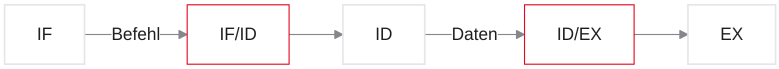
Notizen

Ohne gespeicherte Zwischenergebnisse gibt es keine getrennten Taktzustände. Das Problem ist nicht galvanische Isolation. Eine ältere Instruktion kann den Wert 10 berechnen, während eine jüngere noch den alten Registerwert 99 gelesen hat. An der Registergrenze wird das zusammengehörige Paket übernommen, nicht ein Strom kurz geöffnet.
Kontrollfrage: Öffnet die Grenze den Strom? Nein, sie speichert einen Zustand. Flip-Flop-Bänke halten dieses Paket.

Folie 19 / 85

Die Pipeline-Register (Barrieren)

- Lösung: Pipeline-Register (Flip-Flop-Bänke)
- Wasserdichte Schotten an den Grenzen der fünf Inseln
- Speichern Zwischenergebnisse der vorherigen Stufe
- Namen: IF/ID, ID/EX, EX/MEM, MEM/WB



Notizen

Ein D-Flip-Flop speichert ein Bit nach dem Taktmodell. Die vier Register heißen IF/ID, ID/EX, EX/MEM und MEM/WB. Der Schrägstrich trennt benachbarte Stufen. IF/ID übernimmt regulär an der Taktgrenze und wird bei einem Stall gehalten. Ein beispielhafter Inhalt sind Instruktion, PC-Information und Gültigkeit. Die späteren Register tragen zusätzlich Operanden, rd und Steuersignale.
Kontrollfrage: Hält IF/ID nur, bis ID bereit ist? Nein, es taktet regulär. Der erste Fetch folgt im gemeinsamen Zeitmodell.

Folie 20 / 85

Takt 1: Fetch

- Start des Fließbands im Timing-Raster
- Instruktion 1 in IF
- PC adressiert, IM liefert den Befehl
- Befehl staut sich am IF/ID-Register
- ID, EX, MEM, WB noch leer

```
Takt 1:
[ I1 IF ] -> [ ID - ] -> [ EX - ] -> [ MEM - ] -> [ WB - ]
```

Notizen

Relativer Takt 1 ist der erste betrachtete Fetch. Der PC adressiert den Befehlsspeicher. Bis zum Taktende liegt die Instruktion zur Übernahme in IF/ID bereit. Die anderen Stufen sind im leeren Anfangszustand ungültig, nicht zwingend elektrisch aus. IF bedeutet Instruction Fetch, IM Instruction Memory.
Kontrollfrage: Arbeiten in Takt 1 schon fünf Befehle? Nein. An der nächsten Grenze aktualisieren mehrere Register gleichzeitig.

Folie 21 / 85

Takt 2: Fetch & Decode

- Nächster Clock-Tick öffnet kurz die Schleusen
- Instruktion 1 rutscht nach ID (Decode, Control Unit)
- Instruktion 2 betritt IF
- Zwei Inseln arbeiten parallel an zwei Instruktionen

```
Takt 2:
[ I2 IF ] -> [ I1 ID ] -> [ EX - ] -> [ MEM - ] -> [ WB - ]
```

Notizen

In Takt 2 wird die erste Instruktion in ID decodiert und liest Register. Die zweite wird gleichzeitig in IF geholt. Die Übernahme geschieht an der Taktgrenze, nicht indem Schleusen nacheinander öffnen. Jede Instruktion behält ihren eigenen PC. Decode liest das Befehlswort, das Register File liefert die Operanden, der Opcode ist die Befehlsklasse.
Kontrollfrage: Gehören IF und ID in Takt 2 zur selben Instruktion? Nein, zu zwei verschiedenen. Nach mehreren Takten ist das Band voll.

Folie 22 / 85

Takt 5: Die volle Pipeline

- Fließband auf Betriebstemperatur
- Alle fünf Stufen lückenlos gefüllt
- Instruktionen 5–1 in IF–WB
- Ab diesem Takt: bei jedem Tick ein fertiger Befehl am Ende

```
Takt 5 (volles Fliessband):
[ Instr 5 (IF) ] -> [ Instr 4 (ID) ] -> [ Instr 3 (EX) ]
  -> [ Instr 2 (MEM) ] -> [ Instr 1 (WB) ]
```

Notizen

In Takt 5 eines idealen fortgesetzten Modells steht die älteste Instruktion in WB, die nächsten in MEM, EX, ID und IF. Ab dann kann pro Takt eine Instruktion abschließen. Ein Eintrag in WB schreibt nicht immer ein Register, zum Beispiel bei einem Store. Bei endlichen Folgen leert sich die Pipeline später wieder.
Kontrollfrage: Wie viele verschiedene Instruktionen sind in Takt 5 aktiv? Fünf, wenn das Band voll und gültig ist. Ein Diagramm zeigt das über die Zeit.

Folie 23 / 85

Das Pipeline-Diagramm

- Standardwerkzeug für komplexe Abläufe
- X-Achse: Taktzyklen (Zeit)
- Y-Achse: Programm-Instruktionen (oben nach unten)
- Diagonal: Befehle wandern durch die Stufen

Befehl	1	2	3	4	5	6	7
`add`	IF	ID	EX	MEM	WB		
`sub`		IF	ID	EX	MEM	WB	
`lw`			IF	ID	EX	MEM	WB

Notizen

Horizontal stehen die Takte, vertikal die dynamischen Instruktionen. Drei unabhängige Befehle brauchen sieben Takte bis zum Ende des letzten WB, weil 3 plus 4 gleich 7 ist. Eine Zeile zeigt die Latenz, die Diagonale die Überlappung, die WB-Spalte die Fertigstellungen. Die gezeigten Befehle müssen unabhängig sein, sonst gelten andere Zeiten.
Kontrollfrage: Warum sieben und nicht fünf Takte für drei Befehle? Wegen Füllung und Leerung. Daraus folgt N plus 4.

Folie 24 / 85

Der ideale Durchsatz

- Im Waschsalonvergleich bleibt die Latenz eines Korbs gleich
- Eine echte Pipeline kann die Einzellatenz durch Registeroverhead erhöhen
- Sie erhöht zuerst den Durchsatz
- Ohne Hazards brauchen N Instruktionen und 5 Stufen $N + 4$ Zyklen
- „Speedup immer 5“ gilt nicht

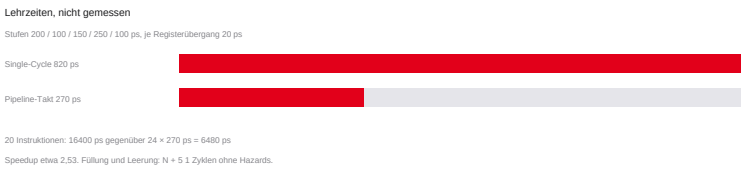


Abbildung 1: Lehrzeiten 820 ps und 270 ps

Notizen

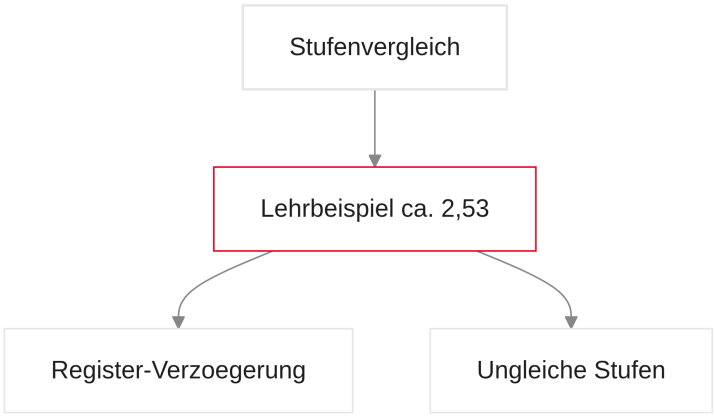
Die Stufenlogik ist 200, 100, 150, 250 und 100 Pikosekunden. Der Overhead von 20 Pikosekunden gilt

einmal je Taktpfad, nicht viermal zusätzlich auf den Single-Cycle-Weg. T single ist 820 ps, T pipe 270 ps. Fünf mal 270 ist 1350 ps Einzellatenz. Für N gleich 20 ist die Pipeline-Taktzahl 24, CPI endlich 1,2. 20 mal 820 ist 16400 ps, 24 mal 270 ist 6480 ps. Der Speedup ist etwa 2,53. ps ist eine Pikosekunde.
Kontrollfrage: Ist die Einzellatenz in der Pipeline kürzer? Nein, 1350 gegen 820. Die nächste Folie erklärt den Abstand zu 5.

Folie 25 / 85

Warum der Faktor nicht 5 ist

- Füllung und Leerung kosten Zyklen
- Registeroverhead und ungleiche Stufen verlängern den Takt der längsten Stufe
- Das Lehrbeispiel oben ergibt für $N = 20$ etwa 2,53
- Keine unbelegte Spanne „typisch 4 bis 4,5“



Notizen

Füllung und Leerung kosten vier zusätzliche Takte, die Stufen sind ungleich, und der Overhead ist 20 ps. Der Grenzwert ohne Hazards ist 820 geteilt durch 270, etwa 3,04, nicht 5. Bei N gleich 1 dauert die Pipeline 1350 ps und der Single Cycle 820 ps. Der Gewinn ist das konkrete 2,53 für 20 Befehle, keine unbelegte Spanne.
Kontrollfrage: Warum nicht Faktor 5? Wegen Ungleichheit, Overhead und endlicher Folge. Als Nächstes müssen die Steuersignale mitwandern.

Folie 26 / 85

Steuersignale in der Pipeline

- Was wird aus der Control Unit (Einheit 6)?
- Steuerwerk sitzt in ID und liest den Opcode
- Beispiel: `MemWrite = 1` für Store
- Problem: Schreiben erst zwei Takte später in MEM
- Lösung: Steuersignale reisen Huckepack mit dem Befehl

Notizen

Die Control Unit in ID erzeugt die Signale des gerade decodierten Befehls. Opcode ist die Familie, nicht jede Operation. MemWrite, die Speicher-Schreibfreigabe, muss zum älteren Store gehören, der inzwischen in MEM steht. Ein Store-Paket enthält daher Freigabe, Adresse, Daten und Gültigkeit zusammen.
Kontrollfrage: Darf MEM das MemWrite des gerade in ID stehenden Befehls nutzen? Nein. Der Weg führt über zwei Registergrenzen.

Folie 27 / 85

Das Wandern der blauen Kabel

- Steuersignale in die Pipeline-Register packen
- `MemWrite` wandert ID → ID/EX → EX/MEM
- Erst wenn der Befehl in MEM steht, wirken die Signale am RAM
- Aktion erst am Zielort — nicht früher



Notizen

MemWrite wandert mit Store-Daten, Adresse und Gültigkeit über ID/EX nach EX/MEM. Es ist kein unabhängig wandernder Faden. RegWrite und rd laufen später bis MEM/WB. Die Wirkung tritt in der passenden Stufe an deren Übernahme ein. rd ist die Zielregisternummer.
Kontrollfrage: Darf das MemWrite eines jüngeren Befehls den älteren Store steuern? Nein. Ohne Abhängigkeiten wäre das ausreichend. Nun folgen benötigte Ergebnisse.

Folie 28 / 85

Die perfekte Illusion

- Bisher: Hardware-Utopie — CPI ~ 1, Takt auf Höchststand
- Fatale Annahme: alle Befehle seien unabhängig
- Realität: Code-Zeilen brauchen ständig Daten voneinander
- Nächster Schritt: die Gefahren (Hazards) des Fließbands

```c
/* Illusion: alles unabhaengig */
int a = 5;
int b = 6;
int c = 7;
```

Notizen

Unabhängige C-Zuweisungen kann ein Compiler vereinfachen oder entfernen. Sie sind nicht garantiert drei Maschineninstruktionen. Eine native Folge mit getrennten Zielen wäre add t0, t1, t2 und add t3, t4, t5. CPI nahe 1 gilt stationär bei langen unabhängigen Folgen, nicht wegen unabhängiger Quellzeilen allein. Kontrollfrage: Garantiert drei C-Zeilen drei Maschinenbefehle? Nein. Der nächste Abschnitt betrachtet echte Registerabhängigkeiten.

Folie 29 / 85

Data Hazards & Forwarding

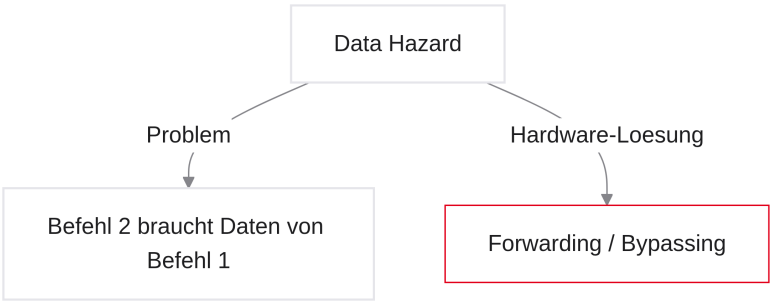
Notizen

Ein Data Hazard ist das zeitliche Problem einer benötigten Datenabhängigkeit. Forwarding leitet ein bereits verfügbares Ergebnis weiter. Die ISA-Wirkung bleibt gleich. Zusätzliche Logik sichert sie trotz Überlappung. Kontrollfrage: Ändert Forwarding das sichtbare Additionsresultat? Nein, nur den Weg. Die nächste Folie baut Produzent und Konsument auf.

Folie 30 / 85

Utopie beenden

- Wir betrachten echten, abhängigen Software-Code
- Definition von Hazards (Gefahren auf dem Fließband)
- Problem: Read-After-Write (RAW)
- Warum Stalling die Performance wieder vernichtet
- Lösung: Forwarding (Bypassing) als Abkürzung



Notizen

RAW bedeutet Read After Write: die jüngere Instruktion braucht den Wert, den die ältere schreiben wird. Ein Stall ist ein Wartezyklus. Er sichert die Korrektheit und senkt den Durchsatz. Er vernichtet nicht automatisch jeden Pipelinegewinn. Die Abhängigkeit steht im Programm. Forwarding und Stall sind zwei Behandlungen. Forwarding nutzt ein Ergebnis vor dem abgeschlossenen Register-Write-Back.
Kontrollfrage: Ist jede RAW-Abhängigkeit ein Stall? Nein. Die Hazard-Arten werden abgegrenzt.

Folie 31 / 85

Was ist ein Hazard?

- Situation, die verhindert, dass der nächste Befehl planmäßig im nächsten Takt starten kann
- Das Fließband muss stottern oder korrigieren
- Drei Kategorien:
- Structural Hazards: zwei Befehle wollen dieselbe Hardware im selben Takt
- Data Hazards: Abhängigkeit von noch nicht fertigen Daten
- Control Hazards: Sprung ändert den PC, während schon falscher Code geladen ist

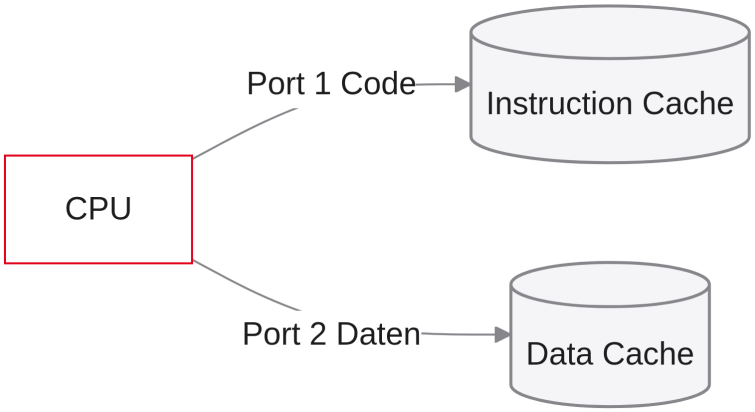
Notizen

Ein Hazard bedroht den korrekten Fortschritt oder den korrekten Kontrollfluss. Die Lösung kann Weiterleitung, Warten oder Verwerfen sein. Ein Structural Hazard ist ein Ressourcenkonflikt, ein Data Hazard eine Datenabhängigkeit, ein Control Hazard ein unbekannter nächster PC. Nicht jeder Data Hazard ist ein Stall.
Kontrollfrage: Muss bei jedem Hazard die Pipeline stehen? Nein. Zuerst ein Ressourcenbeispiel.

Folie 32 / 85

Structural Hazards (Ressourcenkonflikte)

- Befehl in MEM will den RAM schreiben, Befehl in IF will Code lesen
- Gleicher Takt, ein Adress-Port — physikalischer Konflikt
- Lösung: getrennte I/D-Speicher (modified Harvard)
- Instruction Memory und Data Memory (bzw. getrennte L1-Caches)
- In unserem Modell treten Structural Hazards daher praktisch nicht auf



Notizen

Ein gemeinsamer einportiger Speicher kann Fetch und einen Datenzugriff im selben Takt nicht beide bedienen. Ein gemeinsamer Adressraum allein ist noch kein Konflikt, wenn getrennte Ports existieren. Getrennte L1-Instruktions- und Datencaches sind eine mögliche Lösung. L1 ist die erste Cacheebene. Harvard meint hier getrennte Zugriffspfade, nicht eine ISA-Pflicht. Die Registerbank braucht im Modell genug Lese- und Schreibports.
Kontrollfrage: Erzwingt ein gemeinsamer Adressraum den Konflikt? Nein. Als Nächstes geht es um Ergebnisverfügbarkeit.

Folie 33 / 85

Data Hazards (Das Hauptproblem)

- Entstehen durch logische Abhängigkeiten im Quellcode
- Alltag in C: `a = b + c;` dann `d = a + e;`
- `d` braucht das fertige `a`
- Single-Cycle: kein Problem — Zeile 1 ist fertig, bevor Zeile 2 startet
- Pipeline: Überhol-Problem auf dem Fließband

```
/* Klassischer Data Hazard (RAW): */
int a = 10 + 5;   /* schreibt a */
int d = a + 20;   /* liest a sofort danach */
```

Notizen

In-order behält die Programmreihenfolge. Der Konsument braucht nur einen noch nicht zurückgeschriebenen Wert. Das ist kein Überholen. Das C-Bild a gleich b plus c und d gleich a minus e ist konzeptionell, kein garantierter Compileroutput, wenn die Werte nicht konstant sind. RAW bleibt die Lesart nach einem noch ausstehenden Schreiben.
Kontrollfrage: Darf der jüngere Befehl den älteren in der Reihenfolge überholen? In diesem Modell nein. Das Assemblerpaar macht Ziel und Quelle sichtbar.

Folie 34 / 85

Read After Write (RAW) in Assembler

- C-Snippet als RISC-V: `add` gefolgt von `sub`
- `add t0, t1, t2` speichert in `t0`
- `sub t3, t0, t4` liest `t0` sofort
- Konflikt: `sub` verlangt einen Wert, der noch nicht im Register File liegt
- Schreiben erst in WB (Stufe 5), Lesen schon in ID (Stufe 2)

```
add t0, t1, t2    # t0 erst in WB (Stufe 5)
sub t3, t0, t4    # t0 schon in ID (Stufe 2) gelesen
```

Notizen

Lehrwerte: t1 gleich 7, t2 gleich 3, t4 gleich 4, alter t0 gleich 99. add t0, t1, t2 schreibt 10. sub t3, t0, t4 soll 6 ergeben. t0 ist x5, t1 ist x6, t3 ist x28, t4 ist x29. rd ist das Ziel, rs1 und rs2 die Quellen. Das Schreiben liegt in WB, das Lesen in ID. Forwarding kann den alten gelesenen Wert später ersetzen.
Kontrollfrage: Welches Ergebnis ist richtig? 6, nicht 95. Die Zeitachse zeigt beide Befehle.

Folie 35 / 85

Kollision auf dem Fließband

- `add` schreibt `t0` erst in Takt 5 (WB)
- `sub` liest `t0` schon in Takt 3 (ID)
- Katastrophe: `sub` liest veraltete Daten — falsche Rechnung

Befehl	1	2	3	4	5	6
`add t0, t1, t2`	IF	ID	EX	MEM	WB	
`sub t3, t0, t4`		IF	ID	EX	MEM	WB

Notizen

add liegt in IF1, ID2, EX3, MEM4, WB5. sub liegt in IF2, ID3, EX4, MEM5, WB6. Ohne Behandlung könnte sub den alten Wert 99 lesen und 95 rechnen. Das ist ein falscher Datenwert, kein elektrischer Stoß. Die Registernummer und der Datenwert sind verschieden.
Kontrollfrage: Welcher falsche Wert entstünde? 95. Ein Stall könnte das Lesen verzögern.

Folie 36 / 85

Die naive Lösung: Stalling

- Hardware erkennt die Abhängigkeit und hält an (Stall)
- `sub` bleibt in ID, bis `add` WB abgeschlossen hat
- Preis: typisch drei verlorene Taktzyklen (Konvention ohne Write/Read-Split im selben Takt; mit H&H-Registerfile-Hälften oft nur 2 Bubbles)
- Der Speedup durch kürzeren Takt wird teilweise zunichtegemacht

```
ADD:  IF ID EX MEM WB(schreibt)
SUB:     IF .. .. .. ID(liest sicher) EX MEM WB
             ^--- drei Takte Warten ---^
```

Notizen

Zwei Registerlese-Konventionen bleiben getrennt. Schreibt WB und liest ID noch im selben Takt 5, startet sub in EX in Takt 6. Das sind zwei zusätzliche Takte gegenüber der idealen Folge. Kann ID den WB-Wert im selben Takt nicht nutzen, liest sub erst in Takt 6 und rechnet in Takt 7. Das sind drei zusätzliche Takte. Die gehaltene Instruktion bleibt sichtbar in ID. Das ist keine universelle Ripes-Zahl. Harris und Harris ist die Lehrbuchreferenz, falls sie genannt wird.
Kontrollfrage: Ist die Stallzahl immer zwei? Nein, sie hängt von der Lese-Schreib-Konvention ab. Das ALU-Ergebnis existiert schon vor WB.

Folie 37 / 85

Die schlaue Lösung: Forwarding

- Wann ist das `add`-Ergebnis wirklich fertig?
- Nicht erst in WB — die ALU hat es am Ende von EX (Takt 3)
- WB ist nur der bürokratische Speichervorgang
- Idee: Ergebnis direkt am ALU-Ausgang abgreifen und zur nächsten ALU leiten
- Kein Umweg über das Register File

```
Forwarding (Bypassing):
Nutze das Ergebnis, sobald es physikalisch fertig ist.
Warte nicht auf Write-Back.
```

Notizen

Forwarding, auch Bypassing, leitet das Ergebnis weiter. Am Ende von EX ist es berechnet und wird in EX/MEM übernommen. Im nächsten Zyklus kann es den jüngeren EX-Operanden versorgen. WB bestätigt danach den Registerzustand. Es ist nicht nur Bürokratie. Der Endpunkt ist Takt 3, der Verbraucher Takt 4. Kontrollfrage: Muss sub auf das Register-Write-Back warten? In diesem ALU-Fall nein. Der zusätzliche Datenpfad folgt.

Folie 38 / 85

Bypassing (Die Abkürzung)

- Forwarding = Weiterleiten; Bypassing = Umgehen
- Neue Kupferkabel vom EX/MEM-Ausgang zurück an den ALU-Eingang
- MUX wählt die Abkürzung statt des (falschen) Register-Werts
- Konflikt gelöst, ohne einen Takt zu pausieren



Notizen

Der Pfad führt vom EX/MEM-Register zum Operandenmultiplexer der ALU. Gegen die Flussrichtung beschreibt nur die Lage im Bild, nicht die Zeit. Mit Ergebnis 10 und t4 gleich 4 rechnet sub korrekt 6. Der alte ID-Wert wird ersetzt. Nicht jeder Hazard ist damit gelöst.
Kontrollfrage: Welches Ergebnis hat sub? 6. Ein weiter hinten liegender Produzent braucht einen zweiten Pfad.

Folie 39 / 85

Forwarding-Pfade im Datenpfad

- Abkürzungen von allen Stellen, an denen Daten fertig werden
- Pfad 1: EX/MEM → ALU (direkt aufeinanderfolgende Befehle)
- Pfad 2: MEM/WB → ALU (ein unabhängiger Befehl dazwischen)
- Neue Stränge kreuzen das Diagramm von rechts nach links

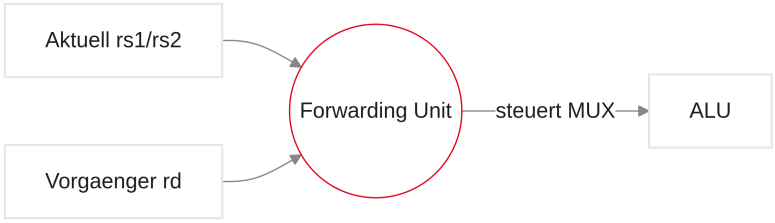
Notizen

EX/MEM liefert ein bereits berechnetes ALU-Ergebnis. MEM/WB liefert den tatsächlichen Write-Back-Wert. Bei einem Load ist das das geladene Wort, nicht die Adresse. Beide Quellen rs1 und rs2 werden geprüft. Ein direkter Vorgänger nutzt EX/MEM. Liegt eine unabhängige Instruktion dazwischen, kann MEM/WB passen. Ein Load in EX/MEM hat dort noch die Adresse.
Kontrollfrage: Darf die Load-Adresse als Nutzdatum weitergereicht werden? Nein. Die Forwarding Unit entscheidet den Pfad.

Folie 40 / 85

Die Forwarding Unit

- Neues Logiknetzwerk vor der EX-Stufe
- Vergleicht Registernummern der Befehle auf dem Fließband
- Frage: Ist `rd` des Vorgängers gleich `rs1`/`rs2` des aktuellen Befehls?
- Wenn ja: MUX auf die Abkürzung schalten



Notizen

Verglichen werden die fünf Bit der Registernummern, nicht die 32-Bit-Daten. Eingänge sind Gültigkeit, RegWrite, Ergebnisart und die Zielnummern beider Quellen. Bei zwei passenden Produzenten gewinnt der jüngere. ForwardA und ForwardB sind getrennte Ausgänge. Ein älterer MEM/WB-Wert ersetzt keinen noch nicht bereiten jüngeren Load.
Kontrollfrage: Gewinnt bei zwei Treffern der ältere Wert? Nein. Die Bedingungen werden ausgeschrieben.

Folie 41 / 85

Die Hazard-Erkennung (Logik)

- Teilbedingung für EX/MEM-Forwarding an rs1:
- `EX/MEM` gültig, `RegWrite = 1`, `rd` ungleich `x0`, `rd` gleich benutztes `rs1`
- Das Ergebnis muss schon der Nutzdatenwert sein, nicht nur eine Load-Adresse
- Bei zwei Treffern hat der jüngere Produzent Vorrang
- Analog für `rs2` und für den MEM/WB-Pfad

```
if (vorheriger_schreibt && sein_ziel == meine_quelle) {
    aktiviere_abkuerzung();
}
```

Notizen

Die EX/MEM-Teilbedingung lautet: der Produzent ist gültig, RegWrite ist 1, rd ist nicht x0, und rd entspricht der benutzten Quelle. Zusätzlich muss der Wert schon das Nutzdatum sein. Ein Load in EX/MEM erfüllt das für das Datum nicht. Der MEM/WB-Pfad prüft dasselbe für den Write-Back-Wert. Der jüngere Treffer hat Vorrang. Ein noch nicht bereiter neuerer Load darf keinen alten Wert durchlassen. Diese Logik ist Forwarding, nicht die spätere Stall-Erkennung. Die Lehrselektoren sind 00, 10 und 01.
Kontrollfrage: Darf x0 einen Bypass auslösen? Nein. Die MUX-Struktur setzt die Wahl um.

Folie 42 / 85

MUX-Erweiterung an der ALU

- Forwarding wählt den Registerdatenwert; `ALUSrc` wählt danach Register oder Immediate
- Lehrcodes: `00` aus ID/EX, `10` aus EX/MEM, `01` aus MEM/WB; `11` unbenutzt
- Diese Codes sind keine Ripes-Portwerte
- Ein Store-Datum darf nicht durch den Immediate ersetzt werden

Notizen

Forwarding wählt zwischen Registerwerten. ALUSrc wählt danach, ob der zweite ALU-Eingang der weitergeleitete Registerwert oder der Immediate ist. 00 ist der aus ID/EX übernommene Registerwert, nicht pauschal Immediate. 10 ist EX/MEM, 01 ist MEM/WB. 11 ist unbenutzt. Das ist eine Lehrkonvention, kein Ripes-Code. Store-Daten bleiben ein eigener Pfad.
Kontrollfrage: Ersetzt Forwarding den Immediate von addi? Nein. Der add/sub-Trace prüft den Pfad.

Folie 43 / 85

Trace: Data Hazard gelöst

- Mit Forwarding: Zeitreise per Bypass
- `sub` bekommt das ALU-Ergebnis rechtzeitig in seiner EX-Stufe
- Kein Taktverlust

Befehl	1	2	3	4	5	6
`add t0, t1, t2`	IF	ID	EX	MEM	WB	
`sub t3, t0, t4`		IF	ID	EX	MEM	WB

Notizen

Rechtzeitige Weiterleitung ersetzt die Metapher Zeitreise. Am Ende von Takt 3 liegt 10 in EX/MEM. In Takt 4 nutzt sub diesen Wert und rechnet 6. Der alte ID-Wert wird ersetzt. Es entsteht keine zusätzliche Bubble. Beide Befehle schreiben später trotzdem ihre Register.
Kontrollfrage: In welchem Takt nutzt sub die 10? In Takt 4, in EX. Der Gewinn gilt nur unter Voraussetzungen, deren Grenze folgt.

Folie 44 / 85

Keine Zeit verloren

- Forwarding rettet den Speedup der 5-Stufen-Pipeline
- Takt bleibt kurz; Software merkt nichts vom Trick
- Zusammenspiel aus Datenpfad-Erweiterung und Kontroll-Logik
- Aber: Ist Forwarding allmächtig?

```
Performance-Check:
Bei RAW von Rechenbefehlen: 0 Taktzyklen Verlust.
CPI bleibt ~1.0
```

Notizen

Null zusätzliche Verlustzyklen gelten für die gezeigte ALU-zu-ALU-Abhängigkeit mit vorhandenem Forwarding. Neue Multiplexer können den kritischen Pfad verlängern. CPI nahe 1 bleibt eine stationäre Aussage. Die endliche Füllung zählt weiterhin. Softwaretransparenz bedeutet gleiche ISA-Ergebnisse, nicht unsichtbare Elektrik.
Kontrollfrage: Ist der Takt durch Forwarding garantiert gleich kurz? Nein. Loads erzeugen ihr Ergebnis später.

Folie 45 / 85

Load-Use Hazards & Stalls

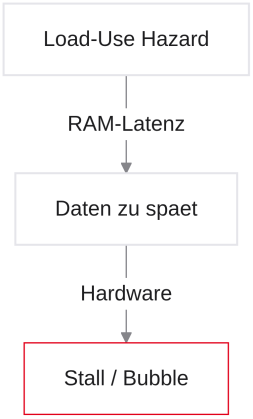
Notizen

Ein Load-Use-Hazard entsteht, wenn die nächste Instruktion das geladene Wort schon in EX braucht. Hazard ist die Abhängigkeit, Stall der Wartezyklus. Das Datum ist nach MEM verfügbar, nicht schon nach der Adressrechnung. Die offene Frage ist, ob die früheste Weiterleitung rechtzeitig kommt.
Kontrollfrage: Ist die Load-Adresse schon das Wort? Nein. Die Lösungen werden benannt.

Folie 46 / 85

Grenzen des Forwarding

- Gefährlichster Datenkonflikt: Load-Use Hazard
- RAM-Latenz sabotiert das Fließband
- Zwangspause: Pipeline-Bubble (NOP)
- Rettung durch Compiler (Instruction Scheduling)



Notizen

Die bisherigen Pfade reichen nicht, weil das Wort im Modell erst nach MEM vorliegt. Eine Bubble ist ein ungültiger Pipeline-Eintrag, keine aus dem Programm geholte NOP. Ein Compiler übersetzt Quelltext. Instruction Scheduling ordnet zulässige Befehle um.
Kontrollfrage: Ist eine Bubble eine addi-Instruktion aus dem Quelltext? Nein. Zuerst die zeitliche Voraussetzung des ALU-Forwardings.

Folie 47 / 85

Ist Forwarding unbesiegbar?

- Voraussetzung: Daten existieren schon irgendwo auf dem Chip
- Bei `add`/`sub`: Ergebnis am Ende von EX fertig
- Nächster Befehl braucht sie in EX — Timing passt
- Frage: Welcher Befehl liefert das Ergebnis erst nach EX?

Notizen

Bereit heißt: der Wert steht am richtigen Pfad rechtzeitig für die Verbraucherberechnung. Ein add hat sein Ergebnis am Ende von EX. Ein lw hat in EX nur die Adresse und das Wort am Ende von MEM. Adresse und Speicherwort bleiben getrennt.
Kontrollfrage: Kann EX/MEM eines Loads das Wort liefern? Nein, dort liegt die Adresse. Das folgende add zeigt die Lücke.

Folie 48 / 85

Der Load-Use Hazard

- Fataler Code: `lw` direkt gefolgt von Nutzung
- `lw t0, 0(s1)` dann `add t3, t0, t2`
- Daten eines `lw` erst am Ende von MEM (Stufe 4)
- Der RAM ist träge

```
lw  t0, 0(s1)     # Daten erst in MEM (Takt 4)
add t3, t0, t2    # braucht Daten schon in EX (Takt 4)
```

Notizen

s1 zeigt auf ein Wort mit dem Lehrwert 42, t2 ist 8. lw t0, 0(s1) und add t3, t0, t2 sollen 50 ergeben. Das Wort liegt im einzykligen MEM erst am Ende von Takt 4 bereit. Der kurze EX-Takt der folgenden Addition kann es nicht mehr verarbeiten. Ein korrektes Modell wartet. Der Code ist nicht fatal.
Kontrollfrage: Welches Ergebnis ist richtig? 50. Das Raster zeigt die fehlende Zeit.

Folie 49 / 85

Warum Forwarding hier versagt

- RAM-Daten erst am Ende von Takt 4 verfügbar
- `add` braucht sie am Anfang von Takt 4
- Ein voller Takt zu spät

Befehl	1	2	3	4	5	6
`lw t0, 0(s1)`	IF	ID	EX	MEM	WB	
`add t3, t0, t2`		IF	ID	EX	MEM	WB

Notizen

Der ungehaltene Plan ist ein Fehlerfall, keine korrekte Ausführung. In Takt 4 endet das Load-MEM, während die Addition schon in EX rechnen müsste. Anfang und Ende sind vereinfachte Stufengrenzen. Es muss genug Zeit bis zur Übernahme bleiben. Eine gemeinsame MEM-und-EX-Rechnung im selben kurzen Takt wäre ein anderer, längerer kritischer Pfad.
Kontrollfrage: Darf add in Takt 4 das Wort nutzen? In diesem Modell nein. Die nächste Folie grenzt die Zeitreise ein.

Folie 50 / 85

Forwarding rückwärts in der Zeit?

- Bypass müsste vom MEM-Ausgang (Ende Takt 4) zum EX-Eingang (Anfang Takt 4)
- Das wäre Datenrücktransport in die Vergangenheit
- Physik verbietet es — harte Wand



Notizen

Ein kombinatorischer Weg vom MEM-Ausgang zum EX-Eingang im selben Takt ist nicht grundsätzlich unmöglich. In diesem Timing reicht die Restzeit nicht. Ein längerer Takt wäre ein anderer Entwurf. Der Balken zeigt Datenbereitstellung am Ende von MEM und die nötige Rechen- und Setup-Zeit. Es fließen Signalzustände, kein Strom rückwärts.
Kontrollfrage: Verbietet die Physik jeden MEM-nach-EX-Draht? Nein. Ein Wartezyklus hält den kurzen Takt.

Folie 51 / 85

Die Hardware-Lösung: Der Stall

- Hazard Detection Unit greift in der ID-Stufe ein
- `lw` fließt normal weiter nach MEM
- Abhängiger Befehl wird in ID eingefroren (kein Wechsel nach EX)
- PC ebenfalls eingefroren (kein neuer Fetch)

```
Erzwungener STALL = Kaffeepause-Takt fuer das Fließband
```

Notizen

Drei Aktionen: den PC halten, IF/ID halten und ID/EX mit einer Bubble belegen. Der Load läuft über EX/MEM und MEM/WB weiter. Der abhängige Befehl bleibt in ID. Kein neuer Fetch übernimmt eine neue Befehlsposition. Am selben PC kann weiter ein Wert anliegen.
Kontrollfrage: Wird der Load eingefroren? Nein. Der leere EX-Platz heißt Bubble.

Folie 52 / 85

Was ist eine Pipeline-Bubble?

- Lücke im Fließband, weil der Folgebefehl auf der Stelle tritt
- EX ist einen Takt leer
- Hardware füllt mit einer Bubble: wirkungsloser NOP
- Bubble wandert EX → MEM → WB und tut nichts

Notizen

Eine Bubble ist ein ungültiger oder wirkungsloser Pipeline-Eintrag. Der abhängige Befehl bleibt in ID erhalten. Im Trace liegt die Bubble in EX in Takt 4, in MEM in Takt 5 und in WB in Takt 6. Eine Anzeige als nop beweist keine aus dem Programm geholte NOP. Eine echte NOP ist addi x0, x0, 0 und schreitet im PC fort.
Kontrollfrage: Verschwindet add durch die Bubble? Nein, es bleibt in ID. Die Nebenwirkungen werden unterdrückt.

Folie 53 / 85

Hardware-Implementierung (NOP)

- Control Unit nullt die Erlaubnis-Pins
- Hazard Unit: `RegWrite = 0`, `MemWrite = 0`, `MemRead = 0`
- Befehl darf nichts Speichern/Schreiben
- Hardware „schläft“ einen Takt

```
NOP in Assembler (Beispiel):
addi x0, x0, 0
# sinnlos, ungefährlich, wirkungslos = Bubble
```

Notizen

Eine Hardware-Bubble kann valid gleich 0 nutzen oder Schreib-, Zugriffs- und Sprungwirkungen abschalten. Die echte NOP addi x0, x0, 0 hat das Wort 0x00000013. Ihr normaler addi-Decoder kann RegWrite setzen. x0 macht den Schreibversuch für Registerdaten wirkungslos. PC und Zähler laufen weiter. Deshalb bedeuten nicht alle Steuersignale null. Eine Bubble schläft nicht und spart nicht automatisch Strom.
Kontrollfrage: Ist NOP gleich alle Signale null? Nein. Der Stall-Trace prüft die Wirkung.

Folie 54 / 85

Load-Use Timeline mit Stall

- `add` einen Takt in der Warteposition
- Bubble füllt die EX-Lücke
- Danach greift klassisches Forwarding (MEM → EX)

Befehl	1	2	3	4	5	6	7
`lw t0, 0(s1)`	IF	ID	EX	MEM	WB		
`add t3, t0, t2`		IF	ID	stall	EX	MEM	WB

Notizen

lw liegt in IF1, ID2, EX3, MEM4, WB5. add liegt in IF2, ID3, gehalten in ID4, EX5, MEM6, WB7. Die Bubble liegt in EX4, MEM5 und WB6. In Takt 5 kommt 42 aus MEM/WB an die EX-Operanden. Mit t2 gleich 8 entsteht 50. t3 übernimmt 50 erst in seinem WB. Der Pfad ist die Weiterleitung nach dem Speicherzyklus, nicht eine beliebige Gleichzeitigkeit.
Kontrollfrage: In welchem Takt rechnet add? In Takt 5. Der Zusatzzyklus verändert den CPI.

Folie 55 / 85

Der Preis des Stalls

- Logisch korrekt, Performance leidet
- CPI steigt (Bubbles verschwenden Takte)
- Viele Arrays laden + sofort verrechnen → viele Blasen
- Hardware am Limit — Rettung durch Software

```
Jeder Load-Use Hazard: typisch 1 Takt Penalty
```

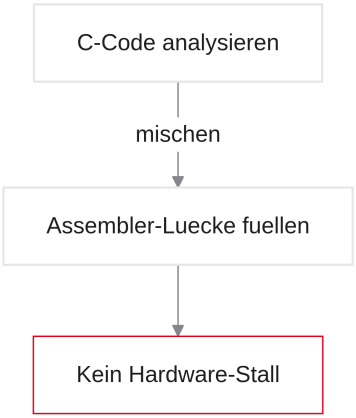
Notizen

Ein zusätzlicher Takt gilt für die unmittelbar folgende EX-Nutzung bei einzykligen Speicherzugriffen und passenden Bypässen. Ein Cache-Miss kann länger dauern. Der CPI-Zuschlag ist der dynamische Load-Use-Anteil, hier zum Beispiel 0,10, also plus 0,10. Die Füllung bleibt separat. Ein Compiler findet nicht immer unabhängige Arbeit.
Kontrollfrage: Kostet jeder Load einen Stall? Nur wenn der Wert sofort in EX gebraucht wird. Scheduling ordnet vorhandene Arbeit um.

Folie 56 / 85

Die Software-Lösung (Compiler)

- Variable nach `lw` nicht sofort im nächsten Befehl nutzen
- Compiler: Instruction Scheduling
- Unabhängige Instruktionen zwischen `lw` und Nutzung schieben



Notizen

Ein Compiler übersetzt Quellcode. Instruction Scheduling verschiebt bereits nötige und zulässige Befehle. Der verschobene Befehl darf Register, Speicher und Kontrollfluss der abhängigen Rechnung nicht zerstören. Produzent, Konsument und eine unabhängige Operation bilden die minimale Abhängigkeit.
Kontrollfrage: Darf jede beliebige Instruktion in die Lücke? Nein. Ein konkretes Beispiel folgt.

Folie 57 / 85

Befehle umsortieren (Scheduling)

- Schlecht (1 Stall):
- `lw` / `add` (abhängig) / `sub` (unabhängig)
- Gut (0 Bubbles):
- `lw` / `sub` (Lücke füllen) / `add` (Daten haben Zeit)

```
# Bubble durch produktive Arbeit ersetzt:
lw  t0, 0(s1)
sub t4, t5, t6    # unabhaengig, fuellt die Luecke
add t3, t0, t2    # Forwarding greift
```

Notizen

Beide Folgen enthalten dieselben lw, add und die unabhängige Operation, nur in anderer Reihenfolge. Die unabhängige Operation darf die Load-Adresse und die add-Operanden nicht überschreiben. Im Modell hat die erste Folge einen Stall und die zweite keinen. Der Unterschied ist ein Zyklus, weil die Instruktionsmenge

gleich bleibt.
Kontrollfrage: Ist eine neue addi-Zeile derselbe Vergleich? Nein. Ein Compiler kann solche Entscheidungen unter Bedingungen automatisieren.

Folie 58 / 85

Die Macht des Compilers

- Generierter Assembler wirkt oft vermischt und unlogisch
- Absicht: Pipelines maximal auslasten
- Mit `-O3` (GCC) aggressive Umsortierungen
- Hardware und Software müssen Hand in Hand arbeiten

Notizen

GCC ist die GNU Compiler Collection. Scheduling ist nicht exklusiv an die Option -O3 gebunden. Zielarchitektur und Optimierungen entscheiden. Es entsteht nicht automatisch stallfreier Code. Die beobachtbare Wirkung muss erhalten bleiben. Ohne einen echten Compilerlauf gibt es keine gemessene Verbesserung.
Kontrollfrage: Garantiert -O3 keine Bubbles? Nein. Auch korrekte Daten lösen nicht die Frage nach dem nächsten PC.

Folie 59 / 85

Control Hazards & Branch Prediction

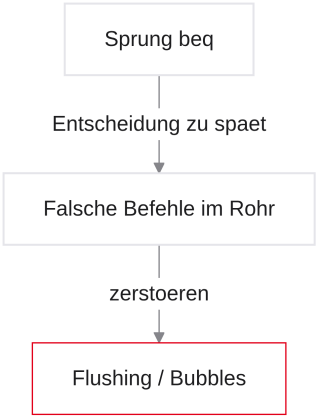
Notizen

Ein Control Hazard ist die Unsicherheit über den korrekten nächsten Kontrollfluss. Branch Prediction ist die Vorhersage des Sprungverhaltens. Ein Branch kann zusätzlich Datenabhängigkeiten haben. Die Abschnitte sind nicht isoliert.
Kontrollfrage: Ist jeder Branch nur ein Datenproblem? Nein. Die bereits geholten jüngeren Befehle motivieren die Frage.

Folie 60 / 85

Der Endgegner des Fließbands

- Was passiert, wenn der PC „lügt“?
- Falsche Fließband-Daten zerstören (Flushing)
- Performance-Einbruch durch Sprünge
- Moderne CPUs als Propheten: Branch Prediction



Notizen

Der PC lügt nicht. Der Fetch arbeitet, bevor das Branch-Ergebnis bekannt ist. Flush verwirft jüngere Instruktionen eines falschen Pfads. Bereits bestätigte richtige Ergebnisse bleiben. Ein Branch und zwei Nachfolgebefehle zeigen das konkreter als eine Symbolkette. Vorhersage und Korrektur gehören zusammen.
Kontrollfrage: Löscht Flush jeden älteren Befehl? Nein. Der zeitliche Informationsmangel folgt.

Folie 61 / 85

Der Control Hazard (`beq`)

- Data Hazards gemeistert — schwerster Konflikt: Kontrollfluss
- Beispiel: `beq t0, t1, ZielLabel`
- Single-Cycle: ALU in Takt 1, PC springt am Taktende
- Pipeline: Fließband steht niemals still
- Während `beq` durch die ersten Stufen kriecht, muss der PC in jedem Takt die nächste Fetch-Adresse wählen

```
Fundamentales Problem:
Der PC muss wissen, wo er weitergeht (Fetch), BEVOR
die CPU verstanden hat, ob sie springen muss!
```

Notizen

beq bedeutet Branch if Equal. Der PC ist die Befehlsadresse. Das Fließband steht bei Stalls sehr wohl. Solange kein Halt oder Redirect greift, holt das Modell sequenzielle Folgeinstruktionen. Es fehlen Bedingung und Zieladresse. Die Branch-Erkennung allein genügt nicht. Der PC der Branch-Instruktion muss bis zur Entscheidung verfügbar bleiben.
Kontrollfrage: Reicht es zu wissen, dass ein Branch vorliegt? Nein. Die Entscheidung liegt im EX-Modell in einem bestimmten Takt.

Folie 62 / 85

Die Latenz der Entscheidung

- Wann fällt die endgültige Sprungentscheidung?
- Subtraktion `t0 - t1` in der Haupt-ALU (Stufe EX, Einheit 5)
- Zero-Flag erst am Ende von Takt 3
- AND vor dem PC gibt die Zieladresse erst zu Beginn von Takt 4 frei

```
beq-Latenz:
Takt 1: IF
Takt 2: ID (Opcode bekannt, Bedingung noch unklar)
Takt 3: EX (ALU rechnet, Zero am Ende)
Takt 4: PC kann umschalten
```

Notizen

Im EX-Modell werden Bedingung und Ziel während Takt 3 berechnet. An der Grenze zu Takt 4 kann der neue PC wirken. Ein AND-Gatter liefert den Auswahlentscheid, nicht selbst die Zieladresse. Zero ist ein internes Gleichheitssignal dieser Variante, kein ISA-Flag. In Ripes müssen Vergleichseinheit und Stufe geprüft werden. Diese Fassung hat Ripes nicht ausgeführt.
Kontrollfrage: Liefert Zero die Zieladresse? Nein. Welche jüngeren Befehle sind dann falsch?

Folie 63 / 85

Die Fließband-Falle

- Zwei Takte lang hat das Fließband blind weitergearbeitet
- Takt 2: Fetch bei $PC + 4$
- Takt 3: Fetch bei $PC + 8$
- Takt 4: Zero = 1 — eigentlich zu `ZielLabel` springen
- Die Befehle in IF und ID sind falsch

Befehl	1	2	3	4
`beq`	IF	ID	EX	MEM
Folge bei $PC + 4$		IF	ID	ungültig
Folge bei $PC + 8$			IF	ungültig
Zielinstruktion				IF

Notizen

Bei einem genommenen EX-Branch werden die bis Takt 3 geholten Folgen an PC plus 4 und PC plus 8 an der Grenze zu Takt 4 ungültig. Sie dürfen in Takt 4 nicht als gültige EX- oder ID-Arbeit erscheinen. In Takt 4 beginnt die Zielinstruktion in IF. Branch-PC, Fetch-PC und Zieladresse bleiben getrennte Werte. Der Offset wird zur Adresse der Branch-Instruktion addiert, nicht zu einem schon weitergelaufenen Fetch-PC. Kontrollfrage: Steht die Folge in Takt 4 noch gültig in EX? Nein. Die Flush-Aktionen folgen.

Folie 64 / 85

Branch Taken: Flushing

- Falsche Befehle in der Röhre müssen vernichtet werden
- Sonst: falsche Variablen, Absturz
- Flush: Steuersignale in IF und ID auf 0
- Falsche Befehle werden zu harmlosen Bubbles (NOPs)
- PC schwenkt auf Label; Fetch startet neu

Notizen

Taken bedeutet genommen. Flush invalidiert im EX-Modell IF/ID und ID/EX und setzt den Ziel-PC. Ältere Inhalte in MEM und WB bleiben. Steuersignale nur in IF und ID auf null zu setzen, beschreibt die Register nicht genau genug. Verworfene Einträge dürfen nicht schreiben und nicht noch einmal umlenken. Ein Absturz folgt daraus nicht automatisch.
Kontrollfrage: Wird der Branch selbst gelöscht? Nein. Die leeren Plätze ergeben die Penalty.

Folie 65 / 85

Der Performance-Verlust (Penalty)

- Erfolgreicher Sprung: typisch 2 Takte Branch Penalty (ohne Optimierung)
- 15–20 % der Befehle sind Branches (If/Else, Schleifen, Calls)
- Ständiges Flushen zerstört den Throughput
- Nächster Schritt: Penalty in Hardware reduzieren

```
Ein Fließband ist nur schnell, wenn es nicht stoppt.
Jeder Flush ist ein Riss im Getriebe.
```

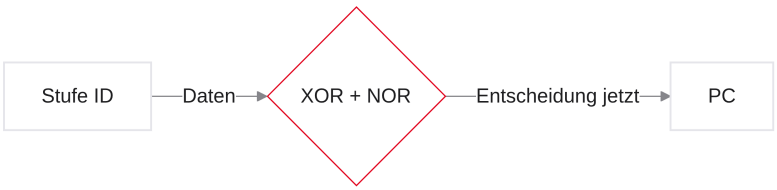
Notizen

Zwei zusätzliche Takte gelten für einen genommenen Branch unter der Annahme Not Taken und EX-Auflösung. Nicht jeder Branch und nicht jede CPU hat diese Penalty. Ein illustratives Beispiel: Branch-Anteil 0,20, Taken-Anteil 0,60, Penalty 2. Der Zuschlag ist 0,24, CPI etwa 1,24. Bei einem Predictor zählt die Fehlvorhersagerate. Calls sind keine bedingten Branches. 15 bis 20 Prozent sind kein gemessener Kurswert.
Kontrollfrage: Kostet ein nicht genommener Branch diese zwei Takte? In diesem Modell nein. Früheres Entscheiden kann den Verlust verkleinern.

Folie 66 / 85

Hardware-Optimierung: Der Komparator

- Branch-Logik nach links schieben (früher entscheiden)
- Extra-Komparator (XOR + NOR) in ID: Register vergleichen, sobald gelesen
- Entscheidung schon in Takt 2
- Penalty sinkt von 2 Takten auf 1 Takt



Notizen

XOR ist exklusives Oder, NOR das negierte Oder. Bitweises XOR und danach NOR kann Gleichheit prüfen. Für bne wird die Gleichheit invertiert. Ein ID-Entscheid braucht aktuelle Operanden, eine frühe Zielrechnung und genug Taktzeit. Forwarding in EX kommt für einen schon in ID benötigten Wert zu spät. Neue Stalls können entstehen. Die Penalty von einem Takt gilt nur, wenn Operanden und Ziel rechtzeitig bereitstehen.
Kontrollfrage: Genügt der Komparator allein? Nein. Bei längerer Unsicherheit wird vorhergesagt.

Folie 67 / 85

Die ultimative Rettung: Branch Prediction

- Moderne Pipelines: oft weit mehr als 5 Stufen (bis ~20)
- Flush würde Dutzende Takte kosten
- Lösung: Branch Prediction (Sprungvorhersage)
- Prozessor rät spekulativ, wohin es geht, und lädt Folge-Befehle

Notizen

Branch Prediction ist die Sprungvorhersage. Richtung und Zielinformation sind verschieden. Tiefe Pipelines können höhere Fehlkosten haben. Die Penalty ist nicht einfach die Stufenzahl. Spekulativer Fetch wird geprüft und korrigiert. Herstellerfrequenzen ohne Quelle bleiben weg. Ultimativ ist diese Technik nicht. Kontrollfrage: Liefert eine Richtungsvorhersage die Zieladresse? Nein. Die einfachste Vorhersage ist der sequenzielle Pfad.

Folie 68 / 85

Statische Vorhersage

- Simpelste Form (z. B. einfache Mikrocontroller)
- Assume Not Taken: immer $PC + 4$ nachladen
- Richtig geraten: null Taktverlust
- Falsch: Flush — im Schnitt oft noch Gewinn

```
Assume Not Taken:
Nehmen Sie an, der Sprung werde nicht genommen,
bis der Zero-Ausgang des Vergleichers Sie eines Besseren belehrt.
```

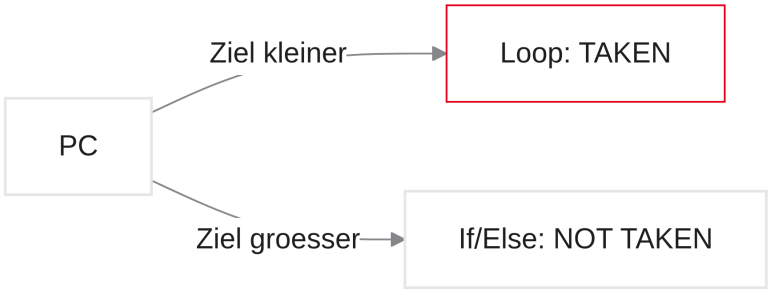
Notizen

Assume Not Taken nimmt PC plus 4 an, solange die C-Erweiterung aus ist. Richtig vorhergesagt bedeutet keine zusätzliche Control-Recovery in diesem Modell. Datenhazards können trotzdem warten. Bei bne ist Nichtnahme nicht einfach Zero gleich 0. Der Gewinn hängt von Taken-Rate und Penalty ab.
Kontrollfrage: Ist Not Taken immer der häufigere Fall? Das ist nicht belegt. Eine Richtungsheuristik nutzt Schleifen.

Folie 69 / 85

Schleifen-Vorhersage (Heuristik)

- Schleifenbranch bleibt rückwärts gerichtet
- Neunmal genommen, einmal nicht genommen: dann Fall-through nach $PC + 4$
- Das letzte Mal ist kein plötzlich vorwärts gerichteter Branch
- Zieladresse $<$ PC $\rightarrow$ Prediction Taken (Schleife)
- Zieladresse $>$ PC $\rightarrow$ Prediction Not Taken (If/Else)



Notizen

Backward Taken, Forward Not Taken ist eine Heuristik. Ein rückwärts gerichteter Countdown-Branch wird neunmal genommen und einmal nicht genommen. Beim letzten Mal fällt er nach PC plus 4 durch. Die Zieladresse bleibt rückwärts. Sie wird nur nicht verwendet. Ein Ziel kleiner als der PC sagt Taken vorher, ein größeres Not Taken. Ein Offset null braucht einen ausdrücklichen Default.
Kontrollfrage: Wird der letzte Schleifenbranch vorwärts? Nein. Dynamische Vorhersage nutzt beobachtete Entscheidungen.

Folie 70 / 85

Dynamische Vorhersage (BHT)

- Dynamic Branch Prediction: lernen zur Laufzeit
- Branch History Table (BHT): Eintrag pro Sprungadresse
- Einfachstes Modell: 1 Bit (letzte Aktion = Prediction)
- Praxis/Vertiefung: oft 2-Bit-Saturating Counter
- Beim Wiedersehen derselben PC-Adresse: Prediction aus der Tabelle

PC-Adresse	Letzte Aktion	Prediction heute
`0x00400100` (`beq`)	Taken	Springen
`0x00400204` (`bne`)	Not Taken	$PC + 4$

Notizen

BHT bedeutet Branch History Table. Der Anfangszustand muss genannt werden. Im Einbitmodell ist die letzte Entscheidung die nächste Vorhersage. Mehrere PCs können denselben Index belegen. Eine Folge Taken, Taken, Taken, Not Taken erzeugt beim letzten Schritt einen Fehler, wenn vorher Taken vorhergesagt wurde. Der Zweibitzähler hat 00 stark Not Taken, 01 schwach Not Taken, 10 schwach Taken und 11 stark Taken. Taken erhöht bis 3, Not Taken verringert bis 0. Das hohe Bit entscheidet. Ein Fehler kippt aus einem starken Zustand zuerst nur die Stärke. Aus einem schwachen Zustand kann ein Ereignis die Richtung wechseln. Die Tabelle liefert keine Zieladresse. Ein BTB, Branch Target Buffer, wäre eine ergänzende Zielinformation.
Kontrollfrage: Kippt ein einzelner Fehler nie die Vorhersage? Nur nicht aus einem starken Zustand. Die Bestätigung der vorläufigen Arbeit folgt.

Folie 71 / 85

Spekulative Ausführung

- Herzstück moderner CPUs (Apple M, AMD Ryzen, Intel Core)
- Vorhersage ist eine weitere Mikroarchitektur; eine Trefferquote wird hier nicht als Messwert behauptet
- Fließband führt Code aus, bevor die Legalität feststeht
- Trivia: Spectre/Meltdown hängen an Spekulation und Caches

```
Ohne Branch Prediction würde ein 5-GHz-Prozessor
durch ständiges Flushen nur einen Bruchteil leisten.
```

Notizen

Spekulativ heißt: vorläufig ausführen, bevor der richtige Kontrollfluss feststeht, nicht bevor die Legalität eines Opcodes feststeht. Vorläufige Ergebnisse werden bestätigt oder verworfen. Out-of-order-Ausführung liegt außerhalb dieses Fünfstufenmodells. Falsche Pfade dürfen den Architekturzustand nicht bestätigen. Cacheinhalte können trotzdem bleiben. Spectre und Meltdown sind eine optionale Einordnung und nicht dieses Labor. Eine 5-Gigahertz-Aussage ohne Messung bleibt weg.
Kontrollfrage: Darf ein verworfener Store den Speicher ändern? Als bestätigte Wirkung nein. Ripes zeigt die einfachen Hazards, nicht automatisch diese Predictor-Architektur.

Folie 72 / 85

Praxis: Hazards live in Ripes

Notizen

Ripes ist ein RISC-V-Lehrsimulator. Die Zeitmodelle sind Vorhersagen für eine konkrete Konfiguration. Drei Ziele sind Forwarding, Load-Use-Stall und Branch-Flush. Eine dynamische BHT wird nicht ungeprüft vorausgesetzt.
Kontrollfrage: Zeigt jede Ripes-Version eine BHT? Das ist nicht belegt. Der Arbeitsablauf folgt.

Folie 73 / 85

Vom Papier in den Simulator

- Ripes auf das 5-Stufen-Modell umstellen
- Bypassing-Leitungen im Datenpfad finden
- Pipeline-Bubbles bei Load-Use beobachten
- Flushing bei Schleifen erleben

Notizen

Version, Prozessor und Layout gehören zu jeder Beobachtung. Zuerst die erwartete Stufenbelegung, dann Portwerte. Farben ergänzen Zahlen und Gültigkeit. Zwischen Versuchen wird zurückgesetzt. Eine Branch-Penalty stammt aus dem gewählten Modell, nicht ungeprüft aus dem Lehrdiagramm. Diese Fassung hat die Oberfläche nicht ausgeführt.
Kontrollfrage: Beweist eine Farbe den Bypass? Nein. Die Modellwahl hat Folgen.

Folie 74 / 85

Ripes-Upgrade (5-Stage)

- Chip-Icon oben links
- `5-Stage Processor` wählen (Standard mit Hazard/Forwarding)
- Mit Forwarding und Hazard-Erkennung: Standard für die Aufgaben
- Ohne Forwarding, aber mit Erkennung: zusätzliche Stalls, Ergebnis bleibt korrekt
- Ohne ausreichende Hazard-Behandlung: falsche Operanden
- Ripes-5-Stage der Installation beobachten und nicht mit dem Drei-Stall-Lehrmodell vermischen
- Processor-Tab: Datenpfad hat sich radikal verändert

Notizen

Das Standardmodell ist die fünfstufige RV32-Pipeline mit Forwarding und Hazard-Erkennung, C-Erweiterung aus. Extended zeigt mehr Steuerung. Ein Modell ohne Forwarding, aber mit Hazard-Erkennung, kann länger warten und trotzdem rechnen. Ohne ausreichende Behandlung sind falsche Ergebnisse ein Merkmal dieser Variante, kein Fehler der Standardkonfiguration. Die Menüposition ist versionsabhängig. Kontrollfrage: Darf man die Variante ohne Hazard-Erkennung als Normalfall nehmen? Nein. Register und Steuerblöcke werden identifiziert.

Folie 75 / 85

Das neue Wimmelbild

- Vier breite Pipeline-Register (IF/ID, ID/EX, EX/MEM, MEM/WB)
- Forwarding-Pfade: Linien von rechts zurück an die ALU-MUX
- Hazard Unit und Forwarding Unit als eigene Boxen



Notizen

IF/ID, ID/EX, EX/MEM und MEM/WB sind die vier Grenzen. Die Forwarding Unit wählt Ergebnisse. Die Hazard Detection Unit erkennt nötige Wartebedingungen. Beispielports sind rd, RegWrite, der Weiterleitungswert und Hold oder Flush. Nicht jede Rückleitung ist MEM/WB. Ohne Screenshot bleibt dies ein Lehrdiagramm. Ein UI-Test liegt nicht vor.
Kontrollfrage: Ist jede Rückleitung das Load-Datum? Nein. Die Historie ist die zweite Ansicht.

Folie 76 / 85

Das Pipeline-Tab in Ripes

- Tab Pipeline: Zeit-Takt-Diagramm wie in den Folien
- Y-Achse: Assembler-Code
- X-Achse: Cycles
- Blöcke IF/ID/EX/MEM/WB zeigen die Lage jedes Befehls

Notizen

Die offizielle Einführung nennt Show stage table, nicht zwingend einen Tab namens Pipeline. Während Run wird laut dieser Dokumentation keine Stufenhistorie aufgezeichnet. Clock oder ein passender Auto-Clock-Modus ist zu prüfen. Zeilen sind dynamische Maschineninstruktionen. Gehalten, gültig, Bubble und verworfen müssen an der Version abgelesen werden.
Kontrollfrage: Zeichnet Run die Stage-Tabelle auf? Laut der genannten Dokumentation nein. Der erste Versuch ist eine ALU-Abhängigkeit.

Folie 77 / 85

Labor-Aufgabe 1: Data Hazard provozieren

- Forwarding testen mit:

```
li  t0, 50
li  t1, 100
add t2, t0, t1    # Ergebnis in t2
sub a0, t2, t0    # Abhaengigkeit von t2
```

- Clock Takt für Takt: wenn `add` in MEM und `sub` in EX

Notizen

Das Programm steht in beispiele/vl7_forwarding.s. addi t0, x0, 50 und addi t1, x0, 100 sind native Befehle. Die Kernfolge ist add t2, t0, t1 und sub a0, t2, t0. Erwartet werden t2 gleich 150 und a0 gleich 100. t0 ist x5, t1 ist x6, t2 ist x7, a0 ist x10. Die echten addi x0, x0, 0 davor und danach sind Programm-NOPs, keine Hazard-Bubbles. Der Kernzähler beginnt beim IF von add.
Kontrollfrage: Ist li immer eine Instruktion? Nein, es kann expandieren. Der kritische Takt zeigt den Bypass.

Folie 78 / 85

Forwarding in Ripes sehen

- Im kritischen Takt zum Processor-Tab
- MUX vor dem ALU-Eingang: Forward-Steuersignal $\neq 0$?
- Maus über Bypass-Kabel: fließt dort 150 $(0\text{x}96)$?
- Dann haben Sie Bypassing nachgewiesen

Notizen

Im Kernmodell ist add in Takt 4 in MEM und sub in EX. rs1 von sub braucht 150, rs2 bleibt 50. Das Ergebnis ist 100. Ein Selektor ungleich null beweist das nur zusammen mit Portname, Quelle, Datenwert und konsumierender Instruktion. Die Lehrcodes 00, 10 und 01 sind nicht die Ripes-Codes. a0 ist architektonisch erst nach dem WB von sub gleich 100.
Kontrollfrage: Ist das ALU-Ergebnis schon der Registerwert? Nein. Der Load liefert sein Wort später.

Folie 79 / 85

Labor-Aufgabe 2: Load-Use Stall erzwingen

- Daten und Code, dann:

```
.data
array: .word 42
.text
la  s1, array
lw  t0, 0(s1)
add a0, t0, t0
```

- Pipeline-Tab: was passiert in Takt 4 oder 5?

Notizen

beispiele/vl7_loaduse.s legt ein Wort 42 an eine ausgerichtete Adresse. la s1, array kann mehrere Befehle erzeugen und liegt außerhalb der Kernfolge. lw t0, 0(s1) und add a0, t0, t0 ergeben t0 gleich 42 und a0 gleich 84. Beide Quellen von add brauchen das Load-Datum. Relativ: Load-EX in Takt 3, add gehalten in ID in Takt 4, add-EX in Takt 5.
Kontrollfrage: Ist 0(s1) ein Immediate von vier Byte? Hier ist der Offset 0. Hold und Bubble werden getrennt.

Folie 80 / 85

Die NOP-Bubble identifizieren

- `add` weigert sich, nach EX zu rutschen
- Raster zeigt grauen/leeren Block: `nop` (Bubble)
- Ein Takt Pause für den langsamen RAM

Notizen

Ein grauer Block ist nicht automatisch eine echte nop-Instruktion. IF/ID wird gehalten, ID/EX wird ungültig, der Load läuft nach MEM. Die Bubble erscheint in EX4, MEM5 und WB6. Die sichtbaren Farben prüft die verwendete Version. Der Wartezyklus folgt aus der Ergebnisverfügbarkeit, nicht aus einem pauschal langsamen RAM.
Kontrollfrage: Holt die Hardware addi x0, x0, 0 als Bubble nach? Nicht als Programmbefehl. Branch verwirft statt zu halten.

Folie 81 / 85

Labor-Aufgabe 3: Control Hazard (Flushing)

- Countdown über zehn Schritte, danach Ende:

```
li t0, 10
Loop:
  addi t0, t0, -1
  bne  t0, zero, Loop
# hier ist t0 = 0; die Schleife ist beendet
```

- Was passiert, wenn `bne` springt und der PC zurückgeht?

Notizen

beispiele/vl7_branch.s zählt von 10 abwärts. addi t0, x0, 10 ist eine klare Maschineninstruktion. Die Schleife führt zehnmal addi und zehnmal bne aus. Neunmal wird genommen, einmal nicht. Danach ist a0 gleich 7. bne bedeutet Branch if Not Equal. Jede Iteration hat zusätzlich RAW von addi nach bne. Ein zweiter Versuch mit bereits bereiten Operanden isoliert den Kontrollfluss. Absolute Taktnummern ohne vollständigen Trace bleiben Lehrannahmen.
Kontrollfrage: Wie oft wird der Branch genommen? Neunmal. Die verworfenen Einträge folgen.

Folie 82 / 85

Flushing in Ripes

- Vordere Stufen haben Code unter der Schleife geladen
- Ripes löscht IF/ID (durchstreichen / ungültig)
- Falsche Befehle werden zu NOP-Bubbles — Branch Penalty live

Notizen

Die Auflösungsstufe und alle betroffenen Register prüft das gewählte Ripes-Modell. Nicht nur IF/ID zu löschen genügt als Behauptung. Ein Wrong-Path-Store darf nicht bestätigt werden. beispiele/vl7_flush.s erwartet bei Gleichheit: Speicherwort bleibt 0, t3 bleibt 0, a0 wird 7. Die verworfenen sw und addi dürfen keine bestätigte Wirkung haben. Die konkrete Penalty ist ohne UI-Test nur die Lehrmodellzahl von zwei zusätzlichen Takten. Beim letzten Not-Taken-bne der Schleife läuft der Fall-through.
Kontrollfrage: Zählen verworfene Befehle als abgeschlossene Programminstruktionen? Nein. Zum Schluss wird dieselbe Arbeit umgeordnet.

Folie 83 / 85

Labor-Aufgabe 4: Compiler-Magie selbst gemacht

- Beide Varianten enthalten dieselben drei Befehle und dieselben Endwerte
- Nur die Reihenfolge von `addi a5, a5, 1` ändert sich
- Ein neu erfundenes `addi` wäre ein anderes Programm

```
Werden Sie zum Compiler: sortieren, bis keine NOPs mehr da sind.
```

Notizen

beispiele/vl7_schedule_a.s und vl7_schedule_b.s enthalten dieselben drei Kernbefehle. A führt lw, add, addi a5 aus. B verschiebt addi a5 zwischen lw und add. a5 ist x15 und startet bei 0. In beiden Fällen enden t0 mit 42, a0 mit 84 und a5 mit 1. A braucht im Lehrfenster acht Takte, B sieben. Eine neu eingefügte Instruktion wäre ein anderes Programm und beweist keinen Gewinn. CPI im Fenster ist 8 durch 3 gegen 7 durch 3, einschließlich Füllung. Der globale Ripes-Zähler enthält Setup und Exit.
Kontrollfrage: Darf man nur B um addi erweitern? Dann ist der Vergleich nicht mehr dasselbe Programm. Die Zusammenfassung trennt Nutzen und Kosten.

Folie 84 / 85

Zusammenfassung Einheit 7

- Pipelining kann den Durchsatz erhöhen; die Einzellatenz hängt vom Modell ab
- Lehrbeispiel: ein Befehl 820 ps im Single Cycle, 1350 ps in der Pipeline
- Für 20 Befehle Speedup etwa 2,53, nicht Faktor 5
- Forwarding, Stall, Scheduling, Flush und Prediction haben je einen eigenen Anwendungsbereich

```
Pipelining-Verständnis trennt Durchsatz von Einzellatenz.
```

Notizen

Pipelining überlappt Instruktionen und kann den Durchsatz erhöhen. Die Einzellatenz kann steigen: 820 gegen 1350 Pikosekunden. Für 20 Befehle ist der Speedup etwa 2,53. Forwarding bedient bereite ALU-Ergebnisse. Ein Load-Use-Stall wartet auf das Wort. Scheduling verschiebt zulässige Arbeit. Flush verwirft falsche Folgebefehle. Prediction reduziert Fehlkosten nur passend zu ihrem Modell.
Kontrollfrage: Ändert Pipelining die Latenz nie? Im Waschsalon nicht, im CPU-Lehrbeispiel schon. Die letzte Folie prüft das Verständnis.

Folie 85 / 85

Q&A und Hands-On

- Ripes: 5-Stage einstellen
- Hazard-Szenarien aus den Folien nachbauen
- Ripes fängt nicht jede falsche Modellwahl ab
- Ergebnisse nur in der Konfiguration mit Forwarding und Hazard-Erkennung erwarten



Notizen

Q&A bedeutet Questions and Answers, Hands-On die praktische Übung. Durchsatz ist die Fertigstellungsrate, Latenz die Dauer eines Auftrags. Ideale fünf Stufen brauchen N plus 4 Takte. Beim Forwarding gewinnt der jüngere passende Produzent. Eine Load-Adresse ist nicht das Wort. Eine Bubble ist kein addi x0, x0, 0 aus dem Programm. Faires Scheduling behält dieselben Befehle und Ergebnisse. Die Programme liegen unter beispiele/vl7_forwarding.s, vl7_loaduse.s, vl7_branch.s, vl7_flush.s sowie vl7_schedule_a.s und vl7_schedule_b.s. Der Simulator garantiert korrekte Ergebnisse nur in der passenden Konfiguration. Die folgende Einheit wird hier nicht vorweg behauptet.
Kontrollfrage: Wie viele Takte braucht eine ideale Folge von N Befehlen? N plus 4. Damit sind Durchsatz, Wartezyklen und falsche Pfade unterscheidbar.